

Material Database Info

 Owner	 Tyson Tran
 Tags	
 Created time	February 23, 2026 11:26 PM

How to Install the MatProtLib Material Database

Step 1: Install Python

If you don't already have Python, download it here: [Python for Windows](#)

-  **CRITICAL:** When you run the downloaded installer, look at the very bottom of the first screen. **You MUST check the box that says "Add python.exe to PATH"** before clicking Install.

Step 2: Install Visual Studio Code

If you don't have a code editor, download VS Code here: [Download VS Code](#)

- *Tip:* Once installed, open VS Code, go to the Extensions tab (the squares icon on the left), and install the official **Python** extension by Microsoft so you can run scripts easily.

Step 3: Install MatProtLib

1. Open Visual Studio Code.
2. Open a new terminal (`Terminal` → `New Terminal` at the top of the screen).
3. Paste the following command and hit Enter:

```
pip install matprotlib
```

Step 4: Verify the Installation

To make sure everything is working, create a new Python file (e.g., `test.py`), paste this code, and run it:

```
import matprotlib as mp  
  
# Fetch Inconel 718 and print its room-temperature density
```

```
inc = mp.Inconel("718")
print(f"Success! Inconel 718 Density: {inc.density} kg/m^3")
```

How to Update MatProtLib

Whenever the database is updated with new materials, you must run this command in your terminal to force your computer to download the newest version:

```
pip install --upgrade --no-cache-dir matprotlib
```

Troubleshooting: "pip is not recognized as an internal or external command"

If you get this error when trying to install, it means Python wasn't added to your environment variables during setup. Here is how to fix it without reinstalling:

1. **Find the Python Installer:** Open the Windows Start menu, type "Add or remove programs," and hit Enter.
2. **Modify the Installation:** Scroll down to find Python (e.g., Python 3.11). Click it and select **Modify** (*Windows 11 users: click the three dots ... next to it first*).
3. **Run the Setup:** In the window that pops up, click **Modify** again. Keep hitting "Next" until you reach the "Advanced Options" screen.
4. **Check the Box:** Make sure the box that says "**Add Python to environment variables**" is checked.
5. **Install/Close:** Click "Install" or "OK" to finish the process.
6. **Restart VS Code: Close Visual Studio Code completely and reopen it.** Your terminal needs to restart to see the new environment variables!

Still having issues? Reach out to Tyson Tran on Teams, send me your specific error message, and I will see if I can get it fixed for you!

How to Use

1. Basic Material Lookup

You can retrieve materials directly from the library using `get_material()`, or by using the new streamlined factory functions for specific alloy families (Aluminum, Steel, Inconel). This must be done before looking up any material info.

Python Code:

```
import matplotlib as mp

#Standard retrieval using the exact database name
alloy = mp.get_material("Inconel 718")

#OR: Use the shortcuts!
inc_alloy = mp.Inconel("718")
alum_alloy = mp.Aluminum("6061")
steel_alloy = mp.Steel("1018 Carbon")
```

Shortcut Chart:

Factory Family	Python Shortcut Call	Translates To
Aluminum	mp.Aluminum("6061")	"Aluminum 6061"
	mp.Aluminum("7075")	"Aluminum 7075"
Inconel	mp.Inconel("625")	"Inconel 625"
	mp.Inconel("718")	"Inconel 718"
Steel	mp.Steel("1018 Carbon")	"1018 Carbon Steel"
	mp.Steel("1045 Carbon")	"1045 Carbon Steel"
	mp.Steel("3140 Low-Alloy")	"3140 Low-Alloy Steel"
	mp.Steel("4140")	"4140 Steel"
	mp.Steel("A286")	"A286 Steel"
Stainless	mp.Stainless("303")	"Stainless Steel 303"
	mp.Stainless("304")	"Stainless Steel 304"
	mp.Stainless("316")	"Stainless Steel 316"
	mp.Stainless("17-4 PH")	"SS 17-4 PH"
Copper	mp.Copper("C101")	"Copper C101"
	mp.Copper("C11000")	"Copper C11000"
	mp.Copper("C17200")	"Copper C17200"
GrCop	mp.GrCop("42")	"GrCop-42"
	mp.GrCop("84")	"GrCop-84"

2. Available Materials

To see a full list of what is currently in the database or scroll down this notion:

Python Code:

```
print(mp.list_materials())
```

3. Constant Properties

For properties that don't fluctuate with temperature—or those where temperature-specific data isn't available—we have listed them as constants in the 'Currently Available Materials' section.

You can use standard dot-notation or the `.get()` method.

Python Code:

```
#Dot notation
```

```
density = alloy.density  
print(f"Density: {density} kg/m^3")
```

```
#.get() method
```

```
poisson = alloy.get("poisson_ratio")  
print(f"Poisson Ratio: {poisson}")
```

4. Temperature-Dependent Queries

For properties like Elastic Modulus or Specific Heat, provide a temperature in Kelvin. The library will interpolate the value based on the underlying data arrays.

Python Code:

```
#Get Yield Strength at a specific engine operating temperature
```

```
temp_k = 800.0  
yield_strength = alloy.get("yield_strength", T=temp_k)  
print(f"Yield Strength at {temp_k}K: {yield_strength / 1e6:.2f} MPa")
```

5. Accessing Metadata

Metadata is for non-numeric or non-interpolated info (like "is it magnetic?" or "what is the carbon content?").

- To get a specific metadata value: Use `get_meta("key_name")`.
- To see all metadata at once: You can access the `.metadata` dictionary attribute directly.

Python Code:

```
#Access a specific metadata piece  
max_temp = alloy.get_meta("max_operating_temp")  
print(f"Safe Operating Limit: {max_temp} K")
```

```
# View all metadata for that material
print(alloy.metadata)
```

6. Accessing Fatigue Curves (S-N Data) (Work In progress not sure this works)

Fatigue curves (Stress vs. Number of cycles) are typically stored as a `FatigueProfile` object within your material.

- **To get the full object:** Use `material.fatigue`.
- **To get a value from the curve:** Most setups use a `get_cycles(stress)` or `get_limit(n_cycles)` method within that profile.

Python Code:

```
#Check if the material has fatigue data for its default condition
cond = alloy.default_condition
if cond in alloy.fatigue:
    # Predict life for a given stress (e.g., 150 MPa)
    stress_pa = 150e6
    cycles = alloy.fatigue.get_cycles(stress_pa)
    print(f"Cycles to failure at 150 MPa: {cycles:.2e}")
else:
    print("No fatigue data registered for this material.")
if cond in alloy.fatigue:
    # Predict the max allowable stress for a target lifespan (e.g., 1,000,000 cycles) at Room
    Temp
    target_cycles = 1e6
    temp_k = 293.15
    stress_limit_pa = alloy.fatigue[cond].get_limit(target_cycles, temp_k)
    print(f"Max stress for {target_cycles:.0e} cycles: {stress_limit_pa / 1e6:.2f} MPa")
else:
    print("No fatigue data registered for this material's current condition.")
```

7. Advanced Usage: Multi-Condition Data

Some materials in **MatPropLib** contain multiple datasets within a single entry to account for different heat treatments or manufacturing conditions. All materials have a default condition and doesn't necessarily need to specify what condition you use but...

1. Instantiating with a Specific Condition

If you know you are working with a specific temper or heat treatment, you can set it when you create the material object using the `type` argument.

Python Code:

```
#Sets the default condition to "Solution Treated" for this specific instance
inc_sol = mp.Inconel("625", type="Solution Treated")
yield_sol = inc_sol.get("yield_strength", T=300)
```

2. Overriding the Condition on the Fly

If you have an existing material object and just want to peek at an alternate condition's data for a single calculation, pass the `condition` argument into the `.get()` method.

Python Code:

```
inc = mp.Inconel("625") # Defaults to "Annealed"
#Grabs the "Solution Treated" data without changing the object's overall default
yield_alt = inc.get("yield_strength", T=300, condition="Solution Treated")
```

8. The Interactive Visual Dashboard

If you want to visually explore how properties behave across temperature ranges or quickly export a dataset for a report, `matprotlib` includes a built-in Streamlit web application.

To start the dashboard, run this single line. It will automatically spin up a local server and open the interface in your default web browser:

Python Code:

```
import matprotlib as mp
# Launch the interactive web UI
mp.launch_dashboard()
```

Note: The dashboard allows you to select any registered material and property, view the interpolated temperature curve, and download the exact numerical data as a CSV.

9. Adding Custom Materials

If you are working with a new material or a specific test batch that isn't in the default registry, you can define and register it yourself in your script. Once registered, it behaves exactly like the built-in alloys and can be accessed via `mp.get_material()`.

NOTE: This registers on your local database does not affect the actual database(if you want to change that go to the bottom of this notion)

Python Code:

```
import matprotlib as mp
import numpy as np

# 1. Instantiate a new blank material
custom_mat = mp.Material(name="Experimental Titanium", category="Metal",
default_condition="Annealed")

# 2. Add properties (can be constants or arrays)
# Constant property:
custom_mat.add_prop("density", 4430.0, "kg/m^3")

# Temperature-dependent property [Temp Array (K), Value Array]:
custom_mat.add_prop("yield_strength", [
    np.array([293.15, 500.0, 800.0]),
    np.array([880e6, 650e6, 400e6])
], "Pa")

# 3. Add to the active registry
mp.add_material(custom_mat)

# 4. Use it!
my_titanium = mp.get_material("Experimental Titanium")
print(f"Density: {my_titanium.density} kg/m^3")
```

Using MatProtLib in MATLAB

MATLAB has a built-in Python engine that can talk to installed packages.

NOTE: You must still download the database on Visual Studio Code before trying to use it on MATLAB

Important Rule for MATLAB:

In Python, you can use our quick dot-notation (e.g., `alloy.density`). MATLAB *does not* support this Python syntax. In MATLAB, you **must** use the explicit `.get()` method.

Step 1: Link MATLAB to Python

First, ensure MATLAB knows where your Python is installed. Type this into your MATLAB Command Window (you usually only need to do this once):

Matlab

```
pyenv('Version', 'system')
```

(If it fails, you may need to point it directly to your `python.exe` path).

Step 2: The MATLAB Script (`.m` file)

Here is how to fetch materials and temperature data directly in MATLAB. Note that you must wrap the results in `double()` to convert the Python numbers into standard MATLAB numbers!

Matlab

```
% 1. Fetch the material (Notice the 'py.' prefix)
% You can use all the same shortcuts like Inconel, Steel, Aluminum, etc.
alloy = py.matprotlib.Inconel('718');

% 2. Read room-temperature properties
% Use the .get() method
density = double(alloy.get('density'));
fprintf('Density: %.2f kg/m^3\n', density);

% 3. Read temperature-dependent properties
% To pass keyword arguments (like T=800), use MATLAB's pyargs() function
temp_K = 800.0;
yield_strength = double(alloy.get('yield_strength', pyargs('T', temp_K)));
fprintf('Yield Strength at %.0fK: %.2f MPa\n', temp_K, yield_strength / 1e6);

% 4. Multi-Condition/Heat Treatments
% Use pyargs() to pass the 'type' argument as well
alloy_sol = py.matprotlib.Inconel('625', pyargs('type', 'Solution Treated'));
```

Currently Available Materials (All property and names will be in the format that the database can read them)

Table of Materials

Material	Default Condition	Other Available Conditions	Constant Properties	Temp Dependent Properties	Metadata	Fatigue Curve Available
<u>Aluminum 6061</u>	T6	None	density melting_point poisson_ratio	cte elastic_modulus specific_heat thermal_conductivity	carbon_content heat_treatable machinability_index magnetic weldability	No

Material	Default Condition	Other Available Conditions	Constant Properties	Temp Dependent Properties	Metadata	Fatigue Curve Available
				ultimate_strength yield_strength		
Aluminum 7075	T6	None	density	cte elastic_modulus shear_strength melting_point specific_heat poisson_ratio thermal_conductivity ultimate_strength yield_strength	carbon_content heat_treatable machinability_index magnetic weldability	None
AlSi10Mg	Stress Relieved / T6	None	density elastic_modulus melting_point poisson_ratio shear_modulus specific_heat thermal_conductivity	cte electrical_resistivity ultimate_strength yield_strength	carbon_content heat_treatable machinability_index magnetic weldability	Yes
Copper C101	Annealed	None	density melting_point	cte elastic_modulus electrical_resistivity poisson_ratio shear_modulus specific_heat thermal_conductivity ultimate_strength yield_strength	carbon_content heat_treatable machinability_index magnetic weldability	Yes

Material	Default Condition	Other Available Conditions	Constant Properties	Temp Dependent Properties	Metadata	Fatigue Curve Available
<u>Copper C11000</u>	Annealed	None	density electrical_resistivity melting_point poisson_ratio shear_modulus	cte elastic_modulus specific_heat thermal_conductivity ultimate_strength yield_strength	carbon_content heat_treatable machinability_index magnetic weldability	Yes
<u>Copper C17200</u>	TH04	None	electrical_resistivity melting_point poisson_ratio shear_modulus	cte density elastic_modulus specific_heat thermal_conductivity ultimate_strength yield_strength	carbon_content heat_treatable machinability_index magnetic weldability	Yes
<u>CuCrZr</u>	SAA	None	cte density melting_point poisson_ratio shear_modulus specific_heat	elastic_modulus electrical_resistivity thermal_conductivity ultimate_strength yield_strength	carbon_content heat_treatable machinability_index magnetic weldability	Yes
<u>GRCop-42</u>	PBF-LB + HIP	None	density melting_point	cte elastic_modulus electrical_resistivity poisson_ratio shear_modulus specific_heat thermal_conductivity	carbon_content heat_treatable machinability_index magnetic weldability	Yes

Material	Default Condition	Other Available Conditions	Constant Properties	Temp Dependent Properties	Metadata	Fatigue Curve Available
				ultimate_strength yield_strength		
<u>GRCop-84</u>	L-PBF / Extruded (Stress Relieved)	None	density melting_point	cte elastic_modulus electrical_resistivity carbon_content poisson_ratio heat_treatable shear_modulus machinability_index specific_heat magnetic thermal_conductivity weldability ultimate_strength yield_strength		
<u>1018 Carbon Steel</u>	Standard	None	density melting_point poisson_ratio	cte elastic_modulus carbon_content electrical_resistivity heat_treatable specific_heat machinability_index thermal_conductivity magnetic ultimate_strength yield_strength	No	
<u>1045 Carbon Steel</u>	Standard	None	melting_point	cte density carbon_content elastic_modulus hardening_temperature_C poisson_ratio magnetic specific_heat tempering_temperature_C thermal_conductivity yield_strength	No	

Material	Default Condition	Other Available Conditions	Constant Properties	Temp Dependent Properties	Metadata	Fatigue Curve Available
<u>3140 Low-Alloy Steel</u>	Annealed	None	density elastic_modulus electrical_resistivity melting_point poisson_ratio shear_modulus specific_heat thermal_conductivity	cte ultimate_strength yield_strength magnetic	carbon_content heat_treatable machinability_index weldability	Yes
<u>4140 Steel</u>	Standard	None	density melting_point poisson_ratio shear_modulus ultimate_strength yield_strength	cte elastic_modulus electrical_resistivity specific_heat magnetic thermal_conductivity weldability	carbon_content heat_treatable machinability_index	Yes
<u>Stainless Steel 303</u>	Annealed	None	density melting_point specific_heat	cte elastic_modulus electrical_resistivity poisson_ratio shear_modulus thermal_conductivity ultimate_strength yield_strength	carbon_content heat_treatable machinability_index magnetic weldability	Yes
<u>Stainless Steel 304</u>	Annealed	None	density melting_point poisson_ratio	cte elastic_modulus specific_heat thermal_conductivity	carbon_content heat_treatable machinability_index magnetic	No

Material	Default Condition	Other Available Conditions	Constant Properties	Temp Dependent Properties	Metadata	Fatigue Curve Available
				ultimate_strength yield_strength		
<u>Stainless Steel 316</u>	Annealed	None	melting_point specific_heat	cte density elastic_modulus electrical_resistivity poisson_ratio shear_modulus thermal_conductivity ultimate_strength yield_strength	carbon_content heat_treatable machinability_index magnetic weldability	
<u>SS 17-4 PH</u>	H900	None	density electrical_resistivity melting_point poisson_ratio shear_modulus specific_heat	cte elastic_modulus thermal_conductivity ultimate_strength yield_strength	carbon_content heat_treatable machinability_index magnetic weldability	
<u>A286 Steel</u>	Standard	None		cte density elastic_modulus melting_point poisson_ratio specific_heat ultimate_strength yield_strength	carbon_content heat_treatable machinability_index magnetic	No

Material	Default Condition	Other Available Conditions	Constant Properties	Temp Dependent Properties	Metadata	Fatigue Curve Available
<u>Inconel 625</u>	Annealed	Solution Treated	density melting_point	cte elastic_modulus electrical_resistivity poisson_ratio machinability_index shear_modulus magnetic specific_heat weldability thermal_conductivity ultimate_strength yield_strength		Yes
<u>Inconel 718</u>	Annealed	Aged	density melting_point specific_heat	cte elastic_modulus electrical_resistivity poisson_ratio machinability_index thermal_conductivity weldability torsional_modulus ultimate_strength yield_strength		No
<u>Graphite</u>	Isomolded /Isotropic	None	density melting_point	cte elastic_modulus electrical_resistivity silicon_content poisson_ratio heat_treatable shear_modulus machinability_index specific_heat magnetic thermal_conductivity weldability ultimate_strength yield_strength		Yes

Aa Material	≡ Default Condition	≡ Other Available Conditions	≡ Constant Properties	≡ Temp Dependent Properties	≡ Metadata	≡ Fatigue Curve Available
<u>Carbon Fiber</u>	Standard Graphitized	None	density melting_point	cte elastic_modulus electrical_resistivity poisson_ratio shear_modulus specific_heat thermal_conductivity ultimate_strength yield_strength	carbon_content machinability_index magnetic wettability	
<u>Fiberglass</u>	Woven Epoxy Laminate	None	density melting_point	cte elastic_modulus electrical_resistivity poisson_ratio shear_modulus specific_heat thermal_conductivity ultimate_strength yield_strength	carbon_content machinability_index magnetic wettability	

Units for Material Properties

All units are in SI units so some of the yields and modulus will be large just to warn you. Individual units are listed below:

1. Mechanical Properties

These properties define how the material handles structural loads and deformation.

- `"density"` **kg/m³** (Mass per unit volume)
- `"yield_strength"` **Pa** (Stress at which plastic deformation begins)
- `"ultimate_strength"` **Pa** (Maximum stress the material can withstand before failing)
- `"elastic_modulus"` **Pa** (Stiffness (Young's Modulus) of the material)
- `"shear_modulus"` **Pa** (Ratio of shear stress to shear strain)
- `"shear_strength"` **Pa** (Maximum shear stress before failure)
- `"poisson_ratio"` **unitless** (Ratio of transverse strain to axial strain)
- `"torsional_modulus"` **Pa** (Resistance to twisting (often used for shafts/turbines))

2. Thermal Properties

These are critical for thrust chamber assembly (TCA) heat transfer and expansion calculations.

- `"thermal_conductivity"` **W/m-K** (Ability of the material to conduct heat)
- `"specific_heat"` **J/kg-K** (Heat energy required to raise temperature by 1K)
- `"melting_point"` **K** Temperature at which the material turns to liquid.

3. Electrical & Specialized Properties

Used for sensors, instrumentation, and composite-specific design.

- `"electrical_resistivity"` **Ω·m** (Resistance to electrical current flow)

Adding New Materials & Updating the Codebase

CRITICAL FOR CONTRIBUTORS

Whenever you push an update to the PyPI database, **you must tell everyone on the team to reinstall the package**. They will not get your new materials automatically! Tell them to run this exact command to bypass their computer's cache:

```
pip install --upgrade --no-cache-dir matprotlib
```

For those who want to take on the fun task of adding more materials (or if we find more accurate data sheets for existing ones), follow this guide:

1. Where the Code Lives

The material database code is currently located in my public GitHub [repository](#) .

- **To change how the database works:** Go to `src/matprotlib/core.py` . This is where the `Material` and `Prop` classes, along with the interpolation math, are located.
 - **To add or edit materials:** Go to `src/matprotlib/__init__.py` . All material data and API shortcuts live directly in `__init__.py` .
 - To edit the graph: Go to `dashboard.py` if you want to edit the dashboard and graph function
 - General info: Go to `pyproject.toml` for version number and packages on the database
-

2. Extracting Data from Graphs

Often, aerospace whitepapers only provide graphs rather than raw data tables. If you need to pull specific data arrays from an image of a graph, I highly recommend using a free graph reader tool like [WebPlotDigitizer](#) . It is a lifesaver for digitizing temperature curves.

3. Adding a New Material

If you want to add a new material, open `__init__.py` and scroll past the factory functions. Use the blank template provided at the top of the material definitions.

The Template:

```
# =====  
=====
```

NEW MATERIAL TEMPLATE

Copy **this** block, uncomment the lines, and fill **in** for each **new material**

Use Ctrl+/ **(or Cmd+/ **in most IDEs to quickly uncomment a highlighted block****

```
# =====  
=====
```

```

# # 1. Initialize the Material Container
# #     Name: Common name (e.g., "Titanium 6Al-4V")
# #     Category: Metal, Ceramic, Composite, Plastic, Superalloy
# #     Default Condition: Annealed, Aged, T6, Solution Treated,
etc.
# new_mat = Material(name="INSERT_NAME_HERE", category="General", default_condition="Standard")

# # -----
# # -----
# # A. MECHANICAL PROPERTIES
# #     Format: [np.array([Temp1, Temp2...]), np.array([Val1, Val2...])]
# #     Note: For constant values, just use a float instead of the lists
# # -----
# # -----
# new_mat.add_prop("density", 0.0, "kg/m^3")

# new_mat.add_prop("yield_strength", [
#     np.array([293.15]), # Temp in Kelvin
#     np.array([0.0])     # Value in Pa
# ], "Pa")

# new_mat.add_prop("ultimate_strength", [
#     np.array([293.15]),
#     np.array([0.0])
# ], "Pa")

# new_mat.add_prop("elastic_modulus", [
#     np.array([293.15]),
#     np.array([0.0])
# ], "Pa")

# new_mat.add_prop("shear_modulus", [
#     np.array([293.15]),
#     np.array([0.0])
# ], "Pa")

```

```

# ], "Pa")

# new_mat.add_prop("poisson_ratio", [
#     np.array([293.15]),
#     np.array([0.0])
# ], "") # Dimensionless

# # -----
# # -----
# # B. THERMAL PROPERTIES
# # -----
# # -----
# new_mat.add_prop("thermal_conductivity", [
#     np.array([293.15]),
#     np.array([0.0])
# ], "W/m-K")

# new_mat.add_prop("specific_heat", [
#     np.array([293.15]),
#     np.array([0.0])
# ], "J/kg-K")

# new_mat.add_prop("cte", [
#     np.array([293.15]),
#     np.array([0.0])
# ], "1/K") # Coeff. Thermal Expansion

# new_mat.add_prop("melting_point", 0.0, "K")

# # -----
# # -----
# # C. ELECTRICAL PROPERTIES (Optional)
# # -----
# # -----
# new_mat.add_prop("electrical_resistivity", [
#     np.array([293.15]),
#     np.array([0.0])

```

```

# ], "Ohm-m")

# # -----
# # D. METADATA (Static info - stays as single values)
# # -----
# new_mat.add_meta("carbon_content", 0.0)
# new_mat.add_meta("machinability_index", 0.0) # 0-100 Scale (1
00 = Free-machining brass)
# new_mat.add_meta("heat_treatable", False)
# new_mat.add_meta("magnetic", False)
# new_mat.add_meta("weldability", "Unknown")

# # -----
# # E. FATIGUE DATA (S-N Curves)
# # Structure: { Temperature_K : [Cycles_Array, Stress_Array
_Pa] }
# # -----
# # new_mat.add_fatigue({
# #     293.15: [ np.array([1e4, 1e5, 1e6]), np.array([0.0, 0.
0, 0.0]) ], # Room Temp
# #     500.00: [ np.array([1e4, 1e5, 1e6]), np.array([0.0, 0.
0, 0.0]) ] # Elevated Temp
# # })

# # -----
# # F. REGISTER (Save to Database)
# # -----
# # _default_registry.add_material(new_mat)

```

(Note: If you are adding a completely new family of metals, e.g., Titanium, consider adding a quick factory function like `def Titanium(grade...):` at the bottom of `__init__.py`

to keep the API clean!)

4. How to Publish Your Changes to PyPI

Once you have added your material and saved the files, you must build and upload the new version to PyPI so the rest of the team can `pip install` it.

Step 1: Bump the Version

Open `pyproject.toml` and increase the version number by one (e.g., from `0.0.10` to `0.0.11`). **Save the file.** PyPI will reject your upload if you forget this step.

Step 2: Go to the Codebase Terminal

Open the terminal in your GitHub Codespace (or local environment). Make sure you are in the main project folder where `pyproject.toml` is located.

```
# Once in the codespace go into the matprotlib file
```

```
cd matprotlib
```

Step 3: Clear Memory and Upgrade Tools

First, delete the old build files to prevent upload errors, then ensure your Python packaging tools are completely up to date. Run these commands:

```
# Delete the old package files
rm -rf dist/
```

```
# Upgrade the Python build tools
pip install --upgrade build twine
```

Step 4: Build and Upload

Now, package up your new code and send it to PyPI:

Bash

```
# Build the new package
python -m build
```

```
# Upload to PyPI (Requires API Token)
python -m twine upload dist/*
```

API Token:

Please Contact Tyson Tran for this token info (949-372-8030)

NOTE: When you paste the token into the terminal it actually won't physically appear just press enter and it should work

Step 5: Notify the Team

Once the upload hits 100%, drop a message in the team chat telling them to update their local environments!

```
pip install --upgrade --no-cache-dir matprotlib
```

Please tell everyone to do it twice until they see the number increase(idk why this happens it something with Pypl)

Installing collected packages: matprotlib

Attempting uninstall: matprotlib

Found existing installation: matprotlib 0.1.8

Uninstalling matprotlib-0.1.8:

Successfully uninstalled matprotlib-0.1.8

Successfully installed matprotlib-0.1.9